

Ciclo de Vida de uma Activity

Professor: Henrique Galati
IFSP - Campus Araraquara
Análise e Desenvolvimento de Sistemas

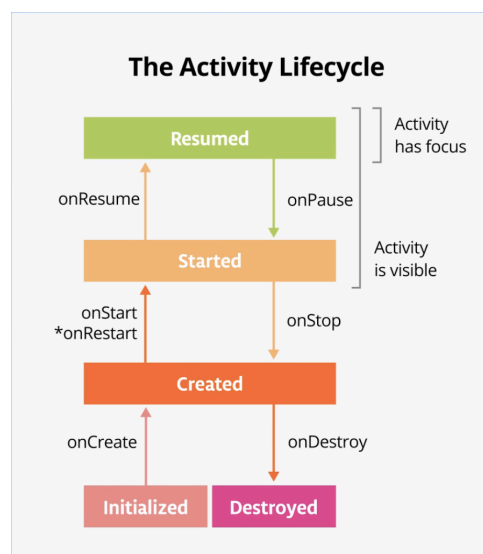
Durante o desenvolvimento de aplicações Android, é comum nos depararmos com um comportamento inesperado: ao **girar o celular (mudança de orientação de tela)**, a tela da Activity atual é reiniciada e os dados exibidos desaparecem. Por que isso acontece?

Isso ocorre porque o sistema Android, ao detectar mudanças de **configuração** (como rotação, mudança de idioma, troca de tema escuro/claro etc.), **deseja recriar a interface para se adaptar à nova configuração**, e para isso ele:

1. **Destrói** a Activity atual;
2. **Cria uma nova instância** da mesma Activity, como se estivesse sendo aberta pela primeira vez.

Por esse motivo, dados carregados dinamicamente ou recebidos por **Intent** podem ser perdidos, caso não sejam preservados corretamente.

O ciclo de vida de uma **Activity** representa os estados que ela percorre **desde sua criação até sua destruição**. Cada estado possui um método de callback que podemos sobrescrever para executar ações específicas. A seguir estão os principais métodos do ciclo de vida (retirados [deste exemplo](#) da documentação):



* indica que **onRestart** não é chamado sempre que o estado muda entre **Created** (criado) e **Started** (iniciado). Ele só é chamado se **onStop()** é chamado e a atividade é reiniciada posteriormente.

Descrição de cada método

Método	Quando é chamado?	Casos de uso recomendados
<code>onCreate()</code>	Quando a Activity está sendo criada pela primeira vez (inclusive após rotação).	Inicializar componentes da interface (View Binding), recuperar dados da <code>Intent</code> , definir listeners e instanciar objetos. Ideal para lógica de configuração.
<code>onStart()</code>	Quando a Activity está prestes a se tornar visível ao usuário.	Iniciar recursos visuais (como animações ou carregamentos de UI). Pode ser usado para registrar listeners de sensores.
<code>onResume()</code>	Quando a Activity entra em primeiro plano e pode interagir com o usuário.	Retomar animações, reiniciar câmera, atualizar UI com dados mais recentes. Ideal para retomar interações suspensas.
<code>onPause()</code>	Quando outra Activity (mesmo que parcialmente visível) toma o foco.	Pausar animações, salvar dados temporários (ex: texto digitado), liberar acesso exclusivo a recursos (como câmera).
<code>onStop()</code>	Quando a Activity não está mais visível.	Liberar recursos pesados (como conexões de rede, sensores ou threads), salvar dados persistentes em banco local.
<code>onDestroy()</code>	Quando a Activity está sendo definitivamente destruída pelo sistema ou por <code>finish()</code> .	Cancelar coroutines, fechar conexões com banco, evitar vazamento de memória (unregister listeners, etc).
<code>onRestart()</code>	Quando a Activity estava parada (<code>onStop</code>) e está prestes a ser iniciada novamente.	Reconfigurar componentes pausados em <code>onStop</code> , reler dados se necessário, reativar monitoramentos temporários.

O que é o Logcat?

Durante o desenvolvimento, é essencial ter uma forma de visualizar o que está acontecendo internamente enquanto a aplicação está sendo executada. Para isso, o Android Studio oferece uma poderosa ferramenta de depuração: o **Logcat**.

O **Logcat** é uma ferramenta que mostra em tempo real os logs (mensagens de registro) gerados pelo sistema Android e pelos aplicativos em execução. Ele exibe informações como:

- Mensagens personalizadas escritas pelo desenvolvedor;
- Logs de erro e falha (crashes);
- Avisos e alertas do sistema;
- Informações de depuração;
- Ciclo de vida de Activities e Fragments (quando logado manualmente).

Como escrever uma mensagem no Logcat?

Para escrever mensagens personalizadas no Logcat, usamos a função **Log** da biblioteca `android.util.Log`.

```
Log.d("MinhaTag", "Mensagem de depuração")
Log.e("MinhaTag", "Algo deu errado!")
```

Exemplo de uso:

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    Log.d("CicloDeVida", "onCreate chamado")
    setContentView(R.layout.activity_main)
}
```

Tipos de mensagens no Logcat

Cada mensagem registrada pode ter um nível de severidade (nível de log), indicado por uma **letra**:

Nível	Letra	Significado	Exemplo de uso
Verbose	V	Mais detalhado possível (e menos performático). Mostra tudo.	<code>Log.v("DEBUG_TAG", "Entrou no método verificarSessao()")</code>
Debug	D	Usado para depuração durante o dev	<code>Log.d("LOGIN", "Email do usuário: usuario@email.com")</code>
Info	I	Informações úteis, não detalhadas demais	<code>Log.i("SYNC", "Sincronização iniciada com sucesso")</code>
Warn	W	Aviso: algo estranho, mas não é erro	<code>Log.w("REDE", "Conexão instável detectada")</code>
Error	E	Erro real: algo deu errado	<code>Log.e("API", "Falha ao buscar dados do servidor")</code>
WTF	WTF	"What a Terrible Failure". Erro crítico inesperado (<i>debug only</i>)	<code>Log.wtf("BANCO", "ID de usuário retornou nulo após login!")</code>

Como visualizar o Logcat no Android Studio?

1. Execute o app em um dispositivo físico ou emulador.
2. No rodapé do Android Studio, clique na aba **"Logcat"**.
3. No campo de filtro:
 - Certifique-se de que o dispositivo e app corretos estão selecionados.
 - Digite sua **Tag personalizada** (ex: `CicloDeVida`) para filtrar.
4. Observe em tempo real as mensagens aparecendo conforme o app roda.



Boas práticas ao usar Logcat

- Use **tags específicas** para cada classe ou componente (ex: "MainActivity", "LoginService"). Você pode utilizar o atributo estático TAG (que fica dentro do **companion object** por padrão)
 - Prefira **Log.d** durante o desenvolvimento e **Log.e** para capturar erros.
 - Nunca deixe logs sensíveis (senha, token, etc.) em produção.
 - Você pode desabilitar logs em builds de produção usando **BuildConfig.DEBUG**.
-

Exemplo Prático: Observando o Ciclo de Vida no Logcat

MainActivity.kt

```
class MainActivity : AppCompatActivity() {  
  
    companion object {  
        private const val TAG = "CicloDeVida"  
    }  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        Log.d(TAG, "onCreate chamado")  
        setContentView(R.layout.activity_main)  
    }  
  
    override fun onStart() {  
        super.onStart()  
        Log.d(TAG, "onStart chamado")  
    }  
  
    override fun onResume() {  
        super.onResume()  
        Log.d(TAG, "onResume chamado")  
    }  
  
    override fun onPause() {  
        super.onPause()  
        Log.d(TAG, "onPause chamado")  
    }  
  
    override fun onStop() {  
        super.onStop()  
        Log.d(TAG, "onStop chamado")  
    }  
}
```

```
override fun onRestart() {
    super.onRestart()
    Log.d(TAG, "onRestart chamado")
}

override fun onDestroy() {
    super.onDestroy()
    Log.d(TAG, "onDestroy chamado")
}
}
```

Execute o app no emulador, abra o **Logcat** e observe os logs gerados a partir das seguintes ações:

- Abra o app: observe a ordem de execução (**onCreate**, **onStart**, **onResume**).
- Aperte o botão Home: veja **onPause**, **onStop**.
- Volte para o app: veja **onRestart**, **onStart**, **onResume**.
- Gire o celular: veja **onPause**, **onStop**, **onDestroy**, depois **onCreate**, **onStart**, **onResume**.
- Feche o app por "Voltar": veja **onPause**, **onStop**, **onDestroy**.

Salvando informações com Bundle: Evitando perda de dados na rotação de tela

No Android, quando o usuário **gira a tela**, o sistema considera essa mudança uma **mudança de configuração** e, por isso, **destroi e recria a Activity**. Isso significa que qualquer dado armazenado apenas em variáveis locais será perdido, a menos que seja salvo manualmente.

Para evitar isso, podemos usar o método **onSaveInstanceState()**, que salva informações em um objeto do tipo **Bundle**, uma estrutura de chave-valor fornecida pelo Android. Este método é chamado **antes da Activity ser destruída**. Nele podemos salvar dados que queremos recuperar após a recriação, como valores de campos, objetos simples, posições de scroll, etc.

Esse Bundle é automaticamente entregue ao método **onRestoreInstanceState()** quando a Activity é recriada, permitindo que você **restaure os dados e continue de onde**

parou. `onRestoreInstanceState()` é chamado **logo após** `onStart()`, caso haja um estado salvo. Usado para restaurar somente o que foi salvo com `onSaveInstanceState()`.

Exemplo prático: Tela de cadastro de contato

Imagine uma tela simples com campos para nome, telefone e email. Ao rotacionar a tela, queremos manter os dados digitados até o momento da rotação.

res/layout/activity_cadastro_contato.xml

```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="24dp">

    <EditText
        android:id="@+id/edtNome"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Nome" />

    <EditText
        android:id="@+id/edtTelefone"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Telefone" />

    <EditText
        android:id="@+id/edtEmail"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Email" />
</LinearLayout>
```

java/ui/CadastroContatoActivity.kt

```
class CadastroContatoActivity : AppCompatActivity() {

    private lateinit var binding: ActivityCadastroContatoBinding
    private lateinit var contato: Contato

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
    }
}
```

```

        binding = ActivityCadastroContatoBinding.inflate(layoutInflater)
        setContentView(binding.root)
    }

    override fun onPause() {
        super.onPause()
        contato = Contato(
            nome = binding.edtNome.text.toString(),
            telefone = binding.edtTelefone.text.toString(),
            email = binding.edtEmail.text.toString(),
            foto = R.drawable.ic_launcher_foreground
        )
    }

    override fun onSaveInstanceState(outState: Bundle) {
        super.onSaveInstanceState(outState)
        outState.putSerializable("contato", contato)
    }

    override fun onRestoreInstanceState(savedInstanceState: Bundle) {
        super.onRestoreInstanceState(savedInstanceState)
        contato = savedInstanceState.getSerializable("contato") as Contato
        setupViews()
    }

    private fun setupViews() {
        binding.edtNome.setText(contato.nome)
        binding.edtTelefone.setText(contato.telefone)
        binding.edtEmail.setText(contato.email)
    }
}

```